

Rapport de projet

Mini application bancaire fictive full stack - MiniBank

Frontend Flutter - Backend Laravel - MySQL Docker

Étudiant	CHEMINAT Theo
Formation	L3 Informatique
Année universitaire	2025-2026
Technologies principales	Flutter, Laravel, MySQL, Docker, Android Studio
Objet du document	Présenter de manière détaillée la conception, l'implémentation, les choix techniques et les limites du projet MiniBank

Sommaire

1. Introduction et objectifs du projet
2. Présentation générale de l'application
3. Environnement de développement et organisation du dépôt
4. Architecture globale et circulation des données
5. Frontend Flutter
6. Backend Laravel
7. Base de données MySQL
8. Fonctionnement détaillé des principales fonctionnalités
9. Assistant MiniBank et évolution du choix technique
10. Crypto simulée
11. Difficultés rencontrées et solutions mises en place
12. Limites du projet et éléments non réalisés
13. Améliorations possibles
14. Bilan personnel et technique
15. Conclusion

Annexes conseillées : captures d'écran, extraits de code, tests

Remarque : des emplacements sont prévus dans ce rapport pour insérer des captures d'écran, un schéma d'architecture et d'éventuels tableaux de test avant export PDF final.

1. Introduction et objectifs du projet

Le projet MiniBank a été réalisé dans un contexte pédagogique dans le but de concevoir une mini application bancaire fictive complète, c'est-à-dire avec une interface mobile, une API backend, une base de données persistante et plusieurs fonctionnalités représentatives d'un petit système transactionnel. L'objectif n'était pas de reproduire un vrai système bancaire en conditions réelles, mais de mettre en pratique des compétences full stack sur un projet cohérent, concret et suffisamment riche pour couvrir à la fois le développement mobile, le développement web backend, la communication réseau, la persistance des données et l'organisation d'un dépôt Git unique.

Le projet a aussi servi de fil conducteur pour manipuler plusieurs outils réellement utilisés dans un contexte professionnel ou semi-professionnel : Flutter pour l'interface mobile, Laravel pour l'API REST, MySQL pour le stockage des données, Docker pour l'isolation de la base, Android Studio pour l'émulateur et VS Code comme environnement principal de développement. Le choix d'un même dépôt GitHub contenant le frontend et le backend a été conservé pour simplifier la gestion de version et garder une vue d'ensemble du projet.

Ce rapport a pour but d'expliquer de façon détaillée ce qui a été réalisé, les décisions techniques qui ont été prises, les difficultés rencontrées, les corrections apportées, les limites de la version actuelle et les évolutions réalistes qui pourraient être envisagées par la suite.

2. Présentation générale de l'application

MiniBank est une application mobile fictive de gestion de compte. Une fois inscrit et connecté, l'utilisateur peut consulter son profil, voir son solde principal, effectuer des opérations simples sur son wallet en euros, consulter son historique, envoyer un virement à un autre utilisateur, utiliser un assistant intégré et manipuler un portefeuille crypto simulé. L'idée générale était de proposer une application qui reste simple à comprendre mais qui fasse intervenir plusieurs briques techniques et plusieurs types de logique métier.

L'application repose sur un modèle client - serveur classique. Le frontend Flutter affiche les écrans et envoie des requêtes HTTP au backend Laravel. Le backend valide les données, applique la logique métier, modifie la base MySQL si nécessaire, puis retourne des réponses JSON au frontend. Les données affichées dans l'application ne sont donc pas seulement locales : elles sont persistées côté base de données et dépendent réellement des opérations effectuées par l'utilisateur.

Bloc fonctionnel	Éléments implémentés	Rôle dans l'application
Authentification	Inscription, connexion, déconnexion, auto-login	Permettre un accès utilisateur persistant et sécurisé à l'application
Wallet principal	Solde, résumé, profil	Afficher la situation financière simulée de l'utilisateur
Opérations bancaires	Dépôt, retrait, historique, virement	Faire vivre le compte et enregistrer les mouvements
Assistant	Chat local côté backend	Répondre à des demandes simples liées au compte
Crypto simulée	Achat, vente, affichage des actifs	Ajouter une dimension supplémentaire de logique métier et de portefeuille

3. Environnement de développement et organisation du dépôt

Le développement a été réalisé principalement sous Windows. Ce point est important car plusieurs difficultés rencontrées pendant le projet sont directement liées à l'environnement local : installation de PHP, activation des extensions nécessaires, configuration de Composer, compatibilité de certains plugins Flutter avec Windows et réglages spécifiques au mode développeur.

- Système principal : Windows
- Éditeur utilisé : Visual Studio Code
- SDK mobile : Flutter installé côté Windows
- Émulation Android : Android Studio avec émulateur Android fonctionnel
- Base de données : MySQL dans Docker via Docker Desktop
- Gestion de version : Git et GitHub Desktop
- Organisation du dépôt : frontend et backend dans le même dépôt GitHub

L'arborescence générale du dépôt est la suivante :

Chemin	Contenu
mobileapp-2026/project/	Application Flutter (frontend mobile)
mobileapp-2026/backend/	API Laravel et logique métier backend
mobileapp-2026/docker-compose.yml	Définition du service MySQL sous Docker
mobileapp-2026/README.md	Instructions courtes de lancement et de test

4. Architecture globale et circulation des données

L'architecture retenue est volontairement classique et lisible. Elle permet de séparer clairement les responsabilités : le frontend gère l'interface et l'expérience utilisateur, le backend centralise la logique métier et les validations, et la base de données se charge de la persistance durable des informations.

Schéma d'architecture globale - MiniBank

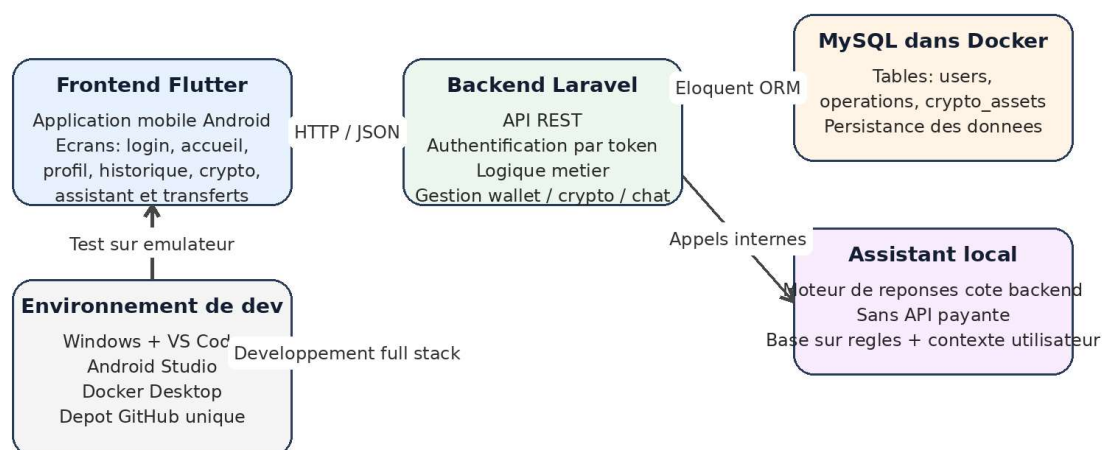


Figure 1 - Schéma d'architecture globale du projet MiniBank

Dans ce modèle, le frontend ne modifie jamais directement la base. Toute opération passe par une route API Laravel. Cela permet d'unifier les contrôles côté serveur : vérification du token, contrôle du solde, recherche de l'utilisateur destinataire, mise à jour du portefeuille crypto, création des lignes d'historique, etc. Ce choix rend le projet plus cohérent qu'une logique majoritairement locale dans Flutter.

La communication entre l'application Flutter exécutée sur l'émulateur Android et le backend local Laravel s'appuie sur l'adresse spéciale 10.0.2.2. Cette adresse est indispensable dans le cas d'un émulateur Android : 127.0.0.1 pointe vers l'émulateur lui-même, alors que 10.0.2.2 redirige vers la machine hôte Windows. Cette distinction a été importante dans la phase de mise au point.

5. Frontend Flutter

Le frontend constitue la partie visible du projet. Il a pour rôle de proposer une navigation claire entre les différentes fonctionnalités, d'envoyer les requêtes au backend, de récupérer les réponses JSON et de maintenir un état cohérent de l'application côté client. L'évolution du frontend a été marquée par un changement important : une première logique locale, pratique pour prototyper, a progressivement été remplacée par une logique réellement connectée à l'API.

5.1 Organisation générale du frontend

Le frontend contient plusieurs écrans correspondant aux usages principaux de l'application : écran de connexion, écran d'inscription, page d'accueil, page profil, page de dépôt, page de retrait, page d'historique, écran de transfert, écran crypto et écran assistant. La page d'accueil joue un rôle central car elle résume la situation du compte et sert de point d'accès vers les autres fonctionnalités.

5.2 Authentification côté Flutter

Lors de la connexion, l'application envoie les identifiants à l'API Laravel. Si la réponse est positive, le token retourné est stocké localement afin d'autoriser un auto-login ultérieur. Ce mécanisme améliore l'expérience utilisateur : l'utilisateur n'a pas besoin de se reconnecter à chaque ouverture de l'application.

Le stockage local du token a demandé de mettre en place une logique de lecture du stockage au démarrage de l'application. Si un token valide est déjà présent, l'utilisateur est redirigé automatiquement vers l'interface principale. Dans le cas contraire, il reste sur l'écran d'authentification.

5.3 Gestion de l'interface et du résumé du wallet

Le résumé du wallet sur l'accueil agrège plusieurs informations : solde principal en euros, informations utilisateur, accès rapides vers les opérations et résumé crypto. L'objectif était d'éviter une interface trop vide tout en gardant une lecture simple. Le frontend joue ici un rôle d'agrégation visuelle, mais l'information de base provient du backend.

5.4 Remplacement progressif de la logique locale

Au début du développement, certaines parties reposaient sur une logique locale dans Flutter, par exemple pour simuler des utilisateurs, des opérations ou des variations de solde. Cette approche est utile pour produire rapidement une démonstration visuelle, mais elle devient problématique dès que l'on veut une cohérence globale. Une opération réalisée sur un écran risquait alors d'être mal répercutée ailleurs ou de disparaître au redémarrage.

Le travail principal côté frontend a donc consisté à nettoyer cette ancienne logique locale et à reconnecter proprement les écrans aux vraies routes API. Ce nettoyage a amélioré la cohérence entre l'affichage, l'état de session et la persistance réelle des données.

Écran de connexion / inscription

Mini Bank

Connectez-vous à votre espace bancaire



Email



Mot de passe

→ Se connecter

+ Créer un compte



Créer un compte

Créer un compte

Entrez vos informations pour créer votre
espace bancaire



Nom



Email



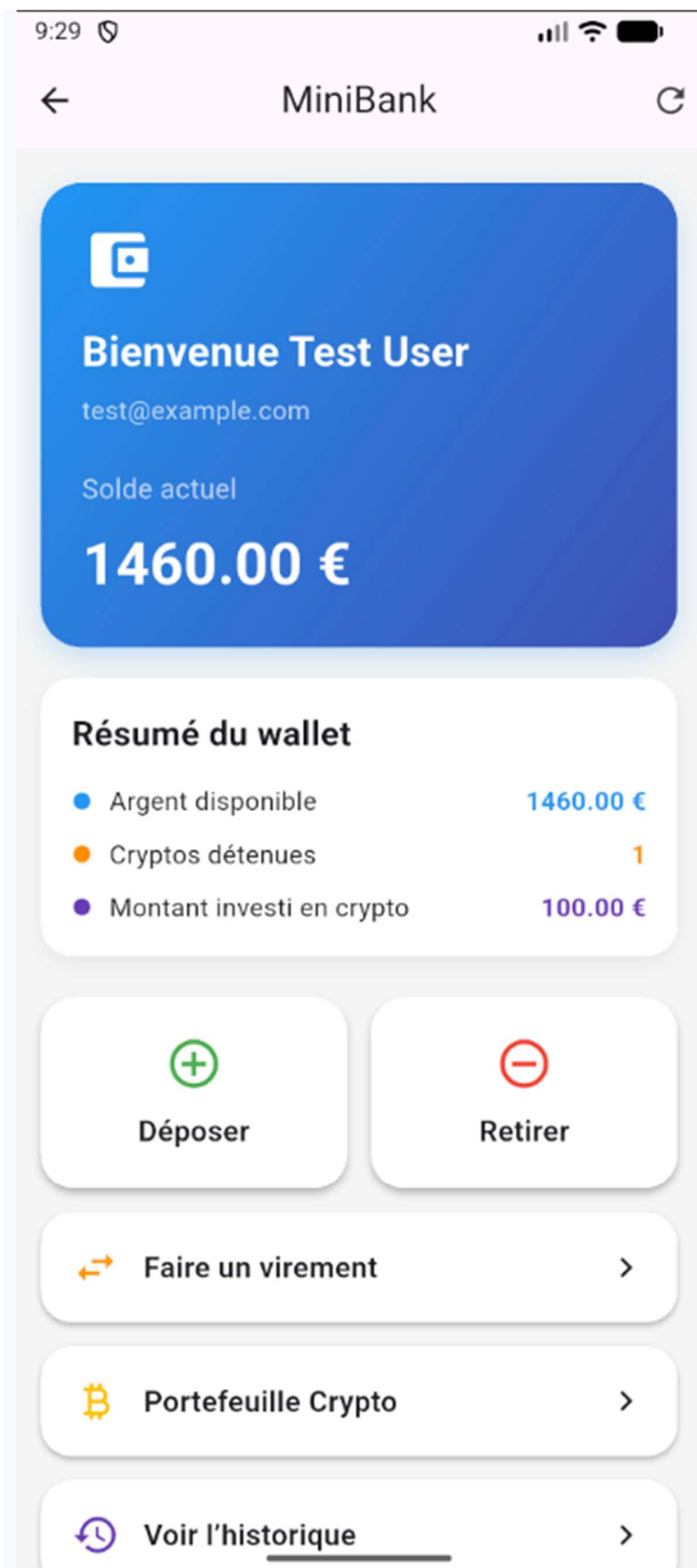
Mot de passe

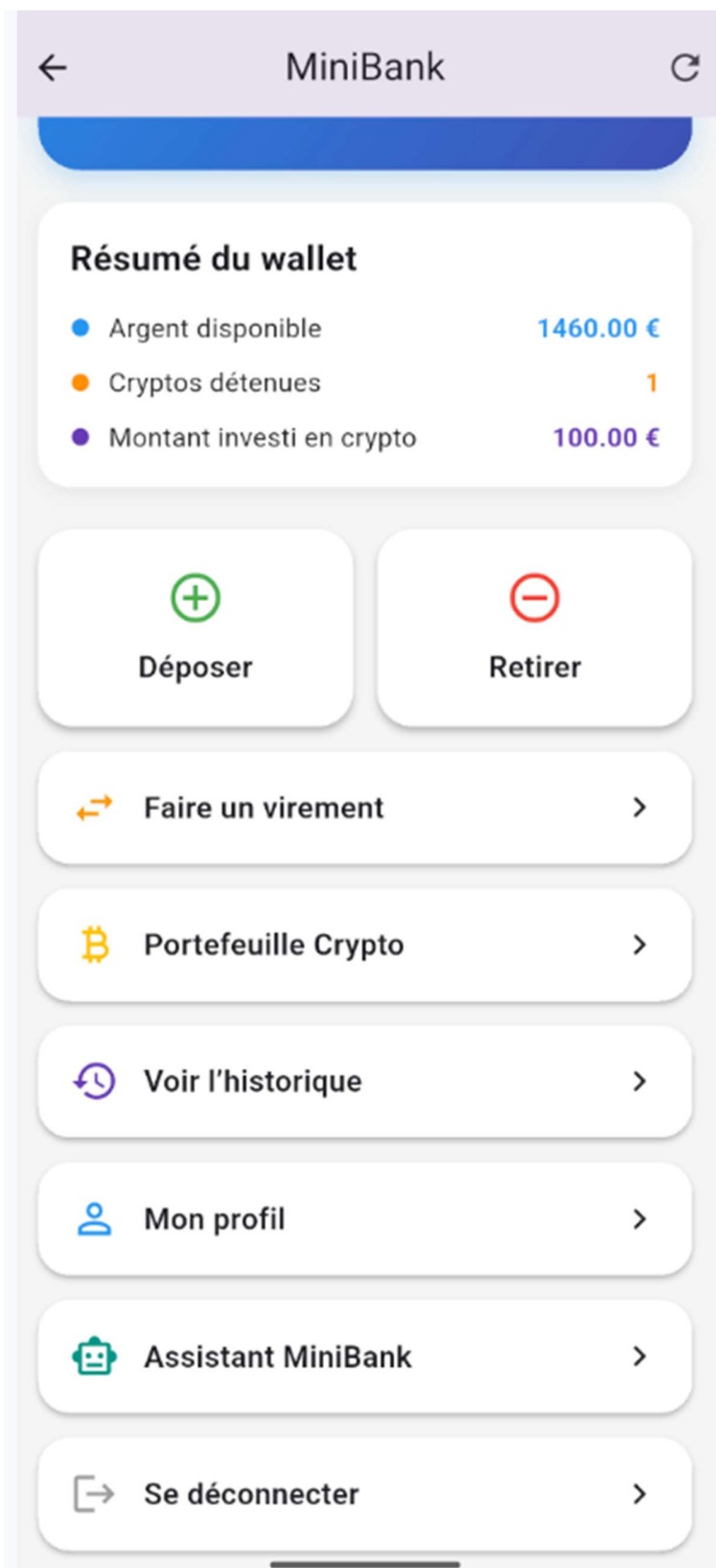


Confirmer le mot de passe



Créer le compte





6. Backend Laravel

Le backend Laravel est le cœur logique du projet. C'est lui qui expose les routes API, authentifie les utilisateurs, applique les règles métier, interagit avec la base MySQL via Eloquent et retourne les données structurées au frontend.

6.1 Routes API principales

Méthode	Route	Fonction
POST	/api/register	Créer un utilisateur
POST	/api/login	Authentifier et retourner un token
GET	/api/profile	Retourner le profil connecté
POST	/api/logout	Invalider ou supprimer la session/token côté client
GET	/api/history	Retourner l'historique des opérations
POST	/api/deposit	Ajouter des fonds au wallet
POST	/api/withdraw	Retirer des fonds du wallet
POST	/api/transfer	Envoyer un virement à un autre utilisateur
GET	/api/crypto	Afficher le portefeuille crypto simulé
POST	/api/crypto/buy	Acheter une crypto simulée
POST	/api/crypto/sell	Vendre une crypto simulée
POST	/api/chat	Envoyer une question à l'assistant MiniBank

6.2 Contrôleurs principaux

Le backend s'organise autour de plusieurs contrôleurs spécialisés.

- AuthController : gère l'inscription, la connexion, la récupération du profil et la déconnexion.
- OperationController : centralise la logique des dépôts, retraits, virements et de l'historique.
- ChatController : gère l'assistant local MiniBank et ses réponses contextuelles.
- CryptoController : implémente l'affichage, l'achat et la vente des actifs crypto simulés.

6.3 Modèles et logique métier

Trois modèles principaux structurent les données : User, Operation et CryptoAsset. Le modèle User stocke l'identité de l'utilisateur ainsi que son solde principal. Operation mémorise les mouvements réalisés sur le compte, ce qui permet de produire un historique lisible. CryptoAsset représente les actifs simulés détenus par l'utilisateur, avec les quantités et les informations nécessaires au calcul du prix moyen d'achat.

Le backend applique plusieurs règles métier importantes : interdiction des retraits ou des achats crypto au-delà du solde disponible, création de traces d'opérations lors des virements, mise à jour atomique du solde avant retour de réponse, contrôle de l'existence du destinataire dans le cas d'un transfert, et agrégation des informations utiles pour l'accueil ou l'assistant.

Exemple de routes API Laravel ou route:list

```
GET|HEAD / ..... routes/web.php:5
POST api/chat ..... Api\ChatController@send
GET|HEAD api/crypto ..... Api\CryptoController@index
POST api/crypto/buy ..... Api\CryptoController@buy
POST api/crypto/sell ..... Api\CryptoController@sell
POST api/deposit ..... Api\OperationController@deposit
GET|HEAD api/history ..... Api\OperationController@history
POST api/login ..... Api\AuthController@login
POST api/logout ..... Api\AuthController@logout
GET|HEAD api/profile ..... Api\AuthController@profile
POST api/register ..... Api\AuthController@register
POST api/transfer ..... Api\OperationController@transfer
POST api/withdraw ..... Api\OperationController@withdraw
GET|HEAD sanctum/csrf-cookie ..... sanctum.csrf-cookie > Laravel\Sanctum > CsrfCookieController@show
GET|HEAD storage/{path} storage.local > vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemServiceProvid...
PUT storage/{path} storage.local.upload > vendor/laravel/framework/src/Illuminate/Filesystem/FilesystemService...
GET|HEAD up ..... vendor/laravel/framework/src/Illuminate/Foundation/Configuration/ApplicationBuilder.php:221

Showing [17] routes
```

7. Base de données MySQL

La persistance des données a été confiée à MySQL exécuté dans un conteneur Docker. Ce choix permet d'isoler la base de données du reste du système, de faciliter les redémarrages et de garder un environnement relativement reproductible.

7.1 Tables principales

Table	Rôle	Exemples d'informations stockées
users	Compte utilisateur	nom, email, mot de passe hashé, solde
operations	Historique du wallet	type d'opération, montant, description, utilisateur lié
crypto_assets	Portefeuille crypto simulé	symbole, quantité, prix moyen d'achat, utilisateur lié

7.2 Relations logiques

Un utilisateur possède plusieurs opérations et peut posséder plusieurs actifs crypto. Cette modélisation est cohérente avec l'usage de l'application : chaque action financière produit un historique et chaque achat crypto modifie un portefeuille lié à un seul utilisateur.

7.3 Migrations et cohérence des données

L'utilisation de migrations Laravel a permis de faire évoluer progressivement la structure de la base : ajout du solde dans la table users, création de la table operations, puis création de la table crypto_assets. Cette démarche est importante car elle évite les changements manuels non tracés et garde l'évolution du schéma dans le dépôt.

8. Fonctionnement détaillé des principales fonctionnalités

8.1 Inscription, connexion et déconnexion

L'inscription crée un nouvel utilisateur avec un mot de passe hashé et un solde initial. La connexion vérifie les identifiants et retourne un token utilisé par le frontend pour authentifier les requêtes suivantes. La déconnexion supprime la session côté application et évite qu'un ancien token soit réutilisé dans un contexte non souhaité.

8.2 Dépôt

Lors d'un dépôt, le backend reçoit un montant, vérifie sa validité, augmente le solde de l'utilisateur, enregistre une opération de type dépôt puis renvoie le nouveau solde. Cette fonctionnalité paraît simple mais elle est essentielle pour valider toute la chaîne front - API - base de données.

8.3 Retrait

Le retrait reprend la même logique générale mais ajoute un contrôle de solde. Le backend doit empêcher qu'un utilisateur passe en négatif. La réponse doit rester claire pour le frontend afin d'afficher un message cohérent en cas d'échec.

8.4 Historique

L'historique récupère les opérations stockées en base pour l'utilisateur connecté. Il constitue la mémoire du compte. Dans le cadre du projet, l'historique n'est pas seulement décoratif : il permet aussi de rendre les fonctionnalités bancaires crédibles et d'offrir une base utile à l'assistant local.

8.5 Virement entre utilisateurs

Le virement repose sur une logique un peu plus riche. L'utilisateur saisit l'email du destinataire et le montant à envoyer. Le backend recherche le destinataire, vérifie qu'il existe et que le solde de l'expéditeur est suffisant, débite l'expéditeur, crédite le destinataire, puis crée des opérations distinctes pour conserver une trace logique des deux côtés. Cette fonctionnalité est intéressante car elle met en jeu plusieurs entités en même temps.

8.6 Wallet principal

Le wallet principal correspond au solde en euros de l'utilisateur. Il est modifié par plusieurs types d'événements : dépôt, retrait, virement envoyé, virement reçu, achat crypto et vente crypto. Le projet a donc nécessité une logique de cohérence transversale : plusieurs fonctionnalités différentes doivent converger vers la même variable métier sans créer d'incohérence.

 Dépôt



Entrer le montant à déposer

 Montant

Valider le dépôt



Retirer



Entrer le montant à retirer

Solde disponible : 1460.00 €

€ Montant

Valider le retrait



Virement



Envoyer de l'argent à un autre utilisateur



Email du destinataire



Montant



Envoyer



Historique

Historique des opérations



Achat crypto

01/04/2026 20:24

Achat de BTC

-100.00 €



Virement reçu

01/04/2026 20:09

Reçu de alice@example.com

+10.00 €



Dépôt

01/04/2026 20:08

Dépôt depuis l'application
mobile

+1500.00 €



Virement envoyé

01/04/2026 20:08

Envoi à alice@example.com

-10.00 €



Virement envoyé

01/04/2026 20:01

Envoi à alice@example.com

-50.00 €



Retrait

18/03/2026 15:41

Premier retrait

-40.00 €

9. Assistant MiniBank et évolution du choix technique

L'assistant MiniBank devait au départ être connecté à une API IA externe afin de produire des réponses plus naturelles et plus ouvertes. Sur le plan de l'idée, cette option était intéressante : elle permettait de donner une dimension plus moderne à l'application et d'ajouter une interaction conversationnelle plus riche.

Dans la pratique, plusieurs problèmes sont apparus. D'abord, l'intégration avec une API externe a soulevé des difficultés de configuration côté backend, notamment sur la gestion des requêtes HTTP sécurisées et des problèmes de SSL ou de cURL. Ensuite, même en corrigeant ces aspects techniques, la contrainte de quota API a rendu la solution peu adaptée à un projet étudiant destiné à être lancé localement, testé par un enseignant et éventuellement réutilisé sans dépendre d'un service payant.

Le choix final a donc été de remplacer cette dépendance par un assistant local côté backend. Concrètement, ChatController interprète certaines demandes simples et retourne des réponses basées sur des règles et sur le contexte utilisateur disponible : solde actuel, opérations récentes, rappels sur le fonctionnement de l'application, questions simples liées au compte ou au wallet.

Ce choix est pertinent pour plusieurs raisons. D'abord, il garantit une démonstration stable sans dépendance externe. Ensuite, il reste cohérent pédagogiquement : l'application propose bien un assistant intégré, mais cet assistant devient une brique contrôlée localement. Enfin, il montre une vraie capacité d'adaptation du projet face à une contrainte réelle, ce qui est souvent plus intéressant qu'une intégration théorique non fonctionnelle.

Écran assistant / exemples de questions-réponses



Assistant MiniBank

Bonjour 🖐️ Je suis l'assistant MiniBank. Vous pouvez me demander votre solde, vos opérations, ou comment utiliser l'application.

Quel est mon solde

Votre solde actuel est de 1460.00 €.

Comment faire un depot

Pour déposer de l'argent, utilisez le bouton 'Déposer' sur l'écran principal.

Écrire un message...



10. Crypto simulée

La partie crypto ajoute une logique métier supplémentaire sans sortir du cadre d'un projet académique local. Le but n'était pas de connecter l'application à un vrai exchange, mais de simuler la détention et l'échange de quelques actifs afin de travailler des calculs de portefeuille et des flux financiers liés au wallet principal.

10.1 Hypothèses de simulation

- Cryptos prises en charge : BTC, ETH et SOL.
- Prix fixes définis côté backend pour simplifier la démonstration.
- Achat : retrait d'euros du wallet principal puis ajout de quantité crypto.
- Vente : réduction de la quantité détenue puis restitution d'euros au wallet principal.
- Calcul du prix moyen d'achat pour garder une vue cohérente du portefeuille.

10.2 Intérêt technique de cette fonctionnalité

Cette partie a demandé de gérer une logique différente des opérations bancaires classiques. Contrairement à un simple dépôt ou retrait, un achat crypto modifie simultanément le wallet principal et la composition d'un portefeuille d'actifs. Il faut donc mettre à jour des tables différentes tout en conservant une lecture claire côté utilisateur.

L'intégration du portefeuille crypto dans le résumé du wallet sur la page d'accueil est également intéressante d'un point de vue frontend : cela montre comment plusieurs sources de données peuvent être synthétisées dans une même interface.

Portefeuille crypto / achat / vente



Portefeuille Crypto

Acheter une crypto

Crypto

BTC



Montant en euros

+ Acheter

Vendre une crypto

Crypto

BTC



Quantité à vendre

💎 Vendre

Mes cryptos



BTC

Quantité : 0.00166667

Prix moyen : 60000.00 €

11. Difficultés rencontrées et solutions mises en place

Cette partie est particulièrement importante car elle montre que le projet ne s'est pas limité à assembler des composants, mais a réellement nécessité des phases de diagnostic, de correction et d'adaptation.

11.1 Installation de PHP et Composer sous Windows

Le backend Laravel a nécessité une installation correcte de PHP et de Composer sur Windows. Cette étape a été moins triviale qu'elle n'en a l'air, notamment à cause des chemins système, de la version de PHP utilisée et de la nécessité de vérifier que les commandes étaient bien reconnues dans le terminal. Une fois PHP et Composer correctement installés et accessibles, il a été possible d'initialiser le backend plus sereinement.

11.2 Activation de pdo_mysql et erreur 'could not find driver'

L'une des erreurs les plus bloquantes a été l'erreur 'could not find driver'. Elle signifiait que Laravel ne pouvait pas utiliser le driver MySQL nécessaire à la connexion PDO. La correction a consisté à activer l'extension pdo_mysql dans la configuration PHP, puis à redémarrer l'environnement concerné. Cette difficulté a été structurante car elle a rappelé qu'une application backend peut sembler correcte côté code tout en restant inutilisable tant que l'environnement PHP n'est pas correctement préparé.

11.3 Connexion Laravel - MySQL Docker

Le fait d'utiliser MySQL dans Docker a imposé une bonne compréhension des ports, du nom du service, du fichier .env Laravel et du moment où les conteneurs sont réellement prêts. Il a fallu vérifier les paramètres DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME et DB_PASSWORD, puis lancer les migrations seulement après validation du bon fonctionnement du conteneur. Cette étape a parfois été compliquée par des erreurs de cache de configuration ou par le fait que la base n'était pas encore disponible au moment de certaines commandes artisan.

11.4 Routes API et fichier routes/api.php

Des soucis sont apparus au niveau de routes/api.php, notamment lorsque certaines routes ne pointaient pas vers les bons contrôleurs, ou lorsque des modifications de structure laissaient des références anciennes. Le diagnostic est passé par route:list et par la vérification explicite de chaque route importante. Cette phase a aussi permis de clarifier l'organisation du backend et d'éviter des incohérences entre le frontend et les endpoints réellement exposés.

11.5 Création manuelle ou correction de contrôleurs

Certains contrôleurs ont dû être créés ou corrigés manuellement, par exemple lorsqu'une référence existait dans les routes mais que la classe correspondante n'était pas présente ou

pas finalisée. Ce type d'erreur est très classique dans un projet en évolution : la structure générale avance plus vite que certains fichiers. La solution a consisté à réaligner les contrôleurs, leurs namespaces, leurs méthodes et leur logique métier.

11.6 Token, logout et problème 'route login not defined'

La gestion du token et de la déconnexion a demandé plusieurs ajustements. Dans certains cas, Laravel renvoyait des comportements inattendus liés à l'authentification, ou des erreurs comme 'route login not defined' lorsque la logique middleware attendait un comportement plus proche d'une application web traditionnelle. Le projet étant une API REST consommée par Flutter, il a fallu adapter les routes protégées et les réponses d'authentification à ce contexte API, avec des réponses JSON cohérentes au lieu de redirections classiques.

11.7 API externe, SSL/cURL et problème de quota

L'intégration initiale de l'assistant avec une API externe a échoué pour deux raisons cumulées : d'une part des difficultés réseau ou SSL/cURL sur l'environnement local, d'autre part une limite de quota peu compatible avec un projet de démonstration. Même lorsque l'idée fonctionnait théoriquement, elle ne garantissait pas une reproductibilité suffisante pour l'évaluation finale.

11.8 Passage à un assistant local

La solution la plus robuste a été de revenir à une logique locale côté backend. Ce changement a nécessité de redéfinir les attentes fonctionnelles de l'assistant : au lieu d'un chatbot généraliste, il est devenu un assistant contextualisé à MiniBank. Cette adaptation a évité de bloquer tout le projet autour d'une seule fonctionnalité annexe.

11.9 Configuration Flutter - Laravel

La connexion entre le frontend Flutter et le backend Laravel a demandé de bien distinguer les contextes d'exécution. Sur émulateur Android, 127.0.0.1 ne permet pas d'atteindre le backend sur Windows ; il faut utiliser 10.0.2.2. Dans d'autres cas, par exemple sur vrai téléphone, il aurait fallu utiliser l'IP locale de la machine. Comprendre cette différence a été indispensable pour éviter de fausses erreurs de connexion.

11.10 Plugins Flutter et Developer Mode Windows

Certains plugins Flutter demandent des prérequis spécifiques sous Windows, notamment l'activation du mode développeur. Sans cela, l'installation ou le fonctionnement de certains composants peuvent poser problème. Cela a rappelé qu'un projet mobile full stack ne dépend pas seulement du code métier, mais aussi de détails de configuration de la machine de développement.

11.11 Nettoyage de l'ancienne logique locale Flutter

Le frontend avait conservé par endroits des traces d'une logique locale utile pour les premiers tests. Une partie du travail a consisté à identifier ces morceaux de code, à éviter les doublons d'état et à reconnecter les écrans aux vraies réponses de l'API. Cette étape a été importante pour éviter des comportements incohérents, par exemple un solde qui semble mis à jour dans une page mais pas dans une autre.

11.12 Cohérence globale frontend / backend

La dernière difficulté n'était pas une erreur unique, mais un enjeu global de cohérence. Chaque fonctionnalité ajoutée devait être réfléchie à la fois côté interface, côté API, côté base et côté scénario utilisateur. Le projet a donc demandé un vrai travail de synchronisation entre plusieurs couches techniques.

12. Limites du projet et éléments non réalisés

Même si le projet est fonctionnel et cohérent dans son périmètre, il reste une application de démonstration académique. Il est donc important de distinguer ce qui a été réellement implémenté de ce qui relèverait d'une version plus avancée ou plus proche d'un produit réel.

- Le projet n'a pas été déployé en ligne dans sa version finale.
- L'application a surtout été pensée pour être testée sur émulateur Android.
- Le vrai support téléphone n'a pas été finalisé pour le rendu.
- Les transactions bancaires sont simulées et ne correspondent à aucun service réel.
- La crypto est entièrement simulée avec des prix fixes et sans source externe.
- Une intégration Stripe ou paiement réel n'a pas été menée à son terme.
- L'assistant final n'est pas connecté à une IA distante dans la version remise.
- La sécurité reste correcte pour un projet étudiant, mais insuffisante pour un usage de production.
- La couverture de tests automatisés reste limitée ou absente selon les parties.

Ces limites ne doivent pas être vues uniquement comme des manques. Elles définissent aussi le cadre du projet : l'objectif principal était de construire une base full stack propre, démontrable localement et suffisamment complète pour illustrer plusieurs compétences techniques.

13. Améliorations possibles

Plusieurs améliorations réalistes pourraient être envisagées pour prolonger le projet.

Axe d'amélioration	Description	Intérêt
Déploiement en ligne	Héberger le backend et la base pour éviter la dépendance au poste local	Rendre l'application testable plus facilement
Support téléphone réel	Adapter la configuration réseau et les tests à un appareil physique	Rendre la démonstration plus proche d'un cas réel
Intégration Stripe	Ajouter un module de paiement réel pour les dépôts	Augmenter le réalisme fonctionnel
Assistant amélioré	Utiliser un moteur plus avancé ou une IA distante avec cache et budget maîtrisé	Rendre les réponses plus naturelles
Prix crypto dynamiques	Connecter l'application à une API de marché ou à un service simulé évolutif	Donner plus de réalisme à la partie portefeuille

Pagination historique	Limiter le volume affiché côté mobile	Améliorer les performances et l'ergonomie
Notifications	Informar l'utilisateur après certaines opérations	Améliorer l'expérience utilisateur
Sécurité renforcée	Validation plus stricte, politiques de rate limiting, journalisation	Mieux préparer un contexte quasi-production
Tests automatisés	Tests unitaires et tests d'API	Sécuriser les évolutions futures
UX / UI	Affiner les messages, le design et les parcours	Rendre l'application plus agréable à utiliser

14. Bilan personnel et technique

Sur le plan technique, ce projet a permis de manipuler un véritable enchaînement full stack, depuis l'interface utilisateur jusqu'à la base de données. Il a obligé à sortir d'une logique purement locale pour construire une application où les actions ont un impact persistant, vérifiable et partagé entre plusieurs fonctionnalités.

Sur le plan méthodologique, le projet a aussi montré l'importance du diagnostic. Plusieurs blocages n'étaient pas liés à l'algorithme lui-même mais à l'environnement, à la configuration ou à la cohérence entre couches. Apprendre à lire une erreur, vérifier les routes, tester les conteneurs, corriger une configuration PHP ou ajuster une URL d'API fait partie intégrante du développement réel.

Enfin, le choix de conserver le frontend et le backend dans un même dépôt a été utile dans ce cadre pédagogique. Cela a facilité la vision d'ensemble et les corrections rapides, même si dans un contexte de production plus grand, une séparation plus stricte pourrait être envisagée.

15. Conclusion

MiniBank est un projet full stack étudiant qui combine une interface mobile Flutter, une API Laravel et une base MySQL sous Docker afin de simuler une petite application bancaire. Malgré un périmètre volontairement fictif, le projet couvre des problématiques concrètes : authentification, opérations sur un wallet, persistance, transferts entre utilisateurs, assistant intégré, portefeuille crypto simulé et cohérence entre plusieurs couches techniques.

Le résultat final n'est pas un produit bancaire réel, mais une démonstration complète et crédible d'un mini système connecté. Le projet est particulièrement intéressant parce qu'il ne s'est pas limité à une réussite linéaire : il a fallu résoudre des erreurs d'environnement, revoir certaines idées initiales et adapter des choix techniques au contexte réel du développement. Cette capacité d'ajustement fait partie des apports les plus utiles du projet.